

MovieRoulette — Project Documentation

Instructor: Nasser Khairy

Team Members & Roles:

Name	Role	Responsibility
Pola Ashraf Amin	Team Leader	Movie Details Page & Random Movie Generator
Ali Nagy Mahmoud	Team Member	About Page
Radwan Elsabah Radwan	Team Member	Favourites Page, Profile Page & Registration (Sign In / Sign Up)
Ibrahim Hassan Ahmed	Team Member	Home Page & Movie List Page
Amir Ashraf Yonan	Team Member	Favourites Page, Profile Page & Registration (Sign In / Sign Up)

1. Project Planning & Management

1.1 Project Proposal

MovieRoulette is a full-stack web application designed to revolutionize movie discovery by combining personalized recommendation algorithms with an intuitive user interface. Modern streaming platforms present users with overwhelming catalogs, leading to decision fatigue. Users often spend excessive time browsing content libraries without making a viewing decision. MovieRoulette addresses this "choice overload" through intelligent filtering and a mood-based random movie generator that prioritizes user preferences while introducing an element of serendipitous discovery.

Scope: * **In-Scope:** User registration/authentication via TMDB, movie/series browsing with granular filters, detailed movie metadata viewing, mood-based random generator, client-side favorites management, responsive design, and global search. * **Out-of-Scope:** In-house user database for content (TMDB is the sole data provider), direct media streaming/playback, and social sharing capabilities.

1.2 Project Plan

The project was executed over a 10-week timeline, organized into 7 distinct phases:

- **Phase 1: Planning & Analysis (Weeks 1-2):** Requirements documentation, system design, and technology selection.

- **Phase 2: Foundation & Setup (Week 3):** Repository initialization, development environment setup, and basic project structure.
- **Phase 3: Core Development (Weeks 4-6):** Authentication, movie browsing, movie details, and basic UI components.
- **Phase 4: Advanced Features (Weeks 7-8):** Random movie generator, favorites system, user profiles, and search functionality.
- **Phase 5: Testing & Refinement (Week 9):** Bug fixing, performance optimization, and usability testing.
- **Phase 6: Documentation & Deployment (Week 10):** Documentation completion, deployment preparation, and final review.
- **Phase 7: Final Presentation:** Presentation preparation and final demonstration.

1.3 Task Assignment

Tasks were distributed to leverage individual strengths while ensuring cohesive integration. The Team Leader (Pola Ashraf Amin) oversaw the architectural decisions and core feature implementations (Details & Random Generator), while team members handled specific domains such as Authentication, Favourites, Home, and UI structure.

1.4 Risk Assessment & Mitigation Plan

Risk	Impact	Mitigation Strategy
TMDB API rate limiting / downtime	High	Implemented request caching, debounced search inputs, and graceful error/loading states (fallback demo data).
Missing/invalid API credentials	High	Added startup validation in <code>server/config.ts</code> to log clear warnings and halt gracefully if keys are missing.
No server-side session store	Medium	Utilized httpOnly cookies with short-lived access tokens and refresh rotation for enhanced security.
Data loss on browser change (Favourites)	Low	Documented as a known system behavior; favorites are stored locally via <code>localStorage</code> to prioritize speed over multi-device sync for the MVP.

1.5 KPIs (Key Performance Indicators)

- **Performance:** Page load times under 3 seconds on a 3G connection; UI interactions respond in under 2 seconds.
- **System Reliability:** 99% uptime for the Node.js API proxy; error rate under 1% via centralized error handlers.
- **Quality:** Over 80% code coverage for critical business logic; zero critical security vulnerabilities.

- **Usability:** 90%+ user satisfaction rate based on usability testing for the random generator feature.
-

2. Literature Review

2.1 Feedback & Evaluation

Instructor: Nasser Khairy

During milestone evaluations, the instructor commended the project's intuitive UI and the robust integration with the TMDB API. Key feedback highlighted the need to ensure robust error handling when TMDB API rate limits are approached, and emphasized optimizing responsive design for mobile users. These suggestions were fully incorporated during Phase 5 (Testing & Refinement) through the implementation of request debouncing and strict CSS media queries.

2.2 Suggested Improvements

Based on post-development technical reviews, the following enhancements have been identified for future iterations: 1. **Backend Testing:** While the frontend has Vitest/Storybook configurations, expanding API-level tests for `server/routes/*` using Supertest would improve backend reliability. 2. **Environment**

Documentation: Inclusion of a `.env.example` file in the repository root to streamline onboarding for new developers. 3. **Cloud Sync for Favourites:** Transitioning from `localStorage` to syncing favorites directly to a user's TMDB account list via the TMDB `/account/{id}/favorite` endpoint.

2.3 Final Grading Criteria

The project evaluation is distributed across the following core criteria: * Requirements & Documentation (20%) * System Architecture & Design (20%) * Implementation & Code Quality (35%) * Testing & Quality Assurance (15%) * Final Presentation & Demonstration (10%)

3. Requirements Gathering

3.1 Stakeholder Analysis

- **End Users (Movie Enthusiasts/Casual Viewers):** Require an intuitive interface, accurate filtering, and rapid, personalized recommendations without decision fatigue.
- **Course Instructor/Evaluators:** Require well-documented code, adherence to software engineering principles, and a functional MVP meeting all academic requirements.
- **TMDB (Data Provider):** Requires proper data attribution and strict adherence to API rate limits and terms of service.

3.2 User Stories & Use Cases

Epic 1: User Authentication * *As a visitor*, I want to sign in with my TMDb account so that I can personalize my experience. * *As a user*, I want to view and manage my profile so that I know my active session details.

Epic 2: Movie Discovery & Browsing * *As a user*, I want to browse trending and top-rated movies/TV shows so I can see what is currently popular. * *As a user*, I want to search for a specific movie by title so that I can quickly find its details.

Epic 3: Random Movie Generator (Core Feature) * *As a user*, I want to generate a random movie suggestion based on my mood (filtered by genre, rating, year) so that I can discover new films effortlessly.

Epic 4: Interaction & Favourites * *As a user*, I want to save movies to a favorites list so that I can easily access them later. * *As a user*, I want to watch trailers and view cast information to help me decide what to watch.

3.3 Functional Requirements

1. The system shall allow users to browse trending, popular, and top-rated movies and TV series.
2. The system shall allow users to search for media by keyword.
3. The system shall generate random movie suggestions based on user-defined filters (genre, rating, language, mood).
4. The system shall display comprehensive movie details (overview, cast, trailers, similar titles).
5. The system shall allow users to toggle movies in their local favorites list.
6. The system shall support TMDb account authentication (Sign In / Sign Out).

3.4 Non-functional Requirements

- **Performance:** All TMDb calls are proxied server-side. The frontend uses code-splitting (`React.lazy` + `Suspense`) and skeleton loaders to maintain low perceived latency.
 - **Security:** API credentials remain strictly server-side. Session IDs are stored in `httpOnly` , `sameSite=lax` cookies.
 - **Usability:** Mobile-first responsive design using Bootstrap 5, featuring accessible ARIA labels and consistent toast notifications for user feedback.
 - **Reliability:** A centralized Express error handler catches exceptions to return consistent JSON errors rather than crashing the Node process.
-

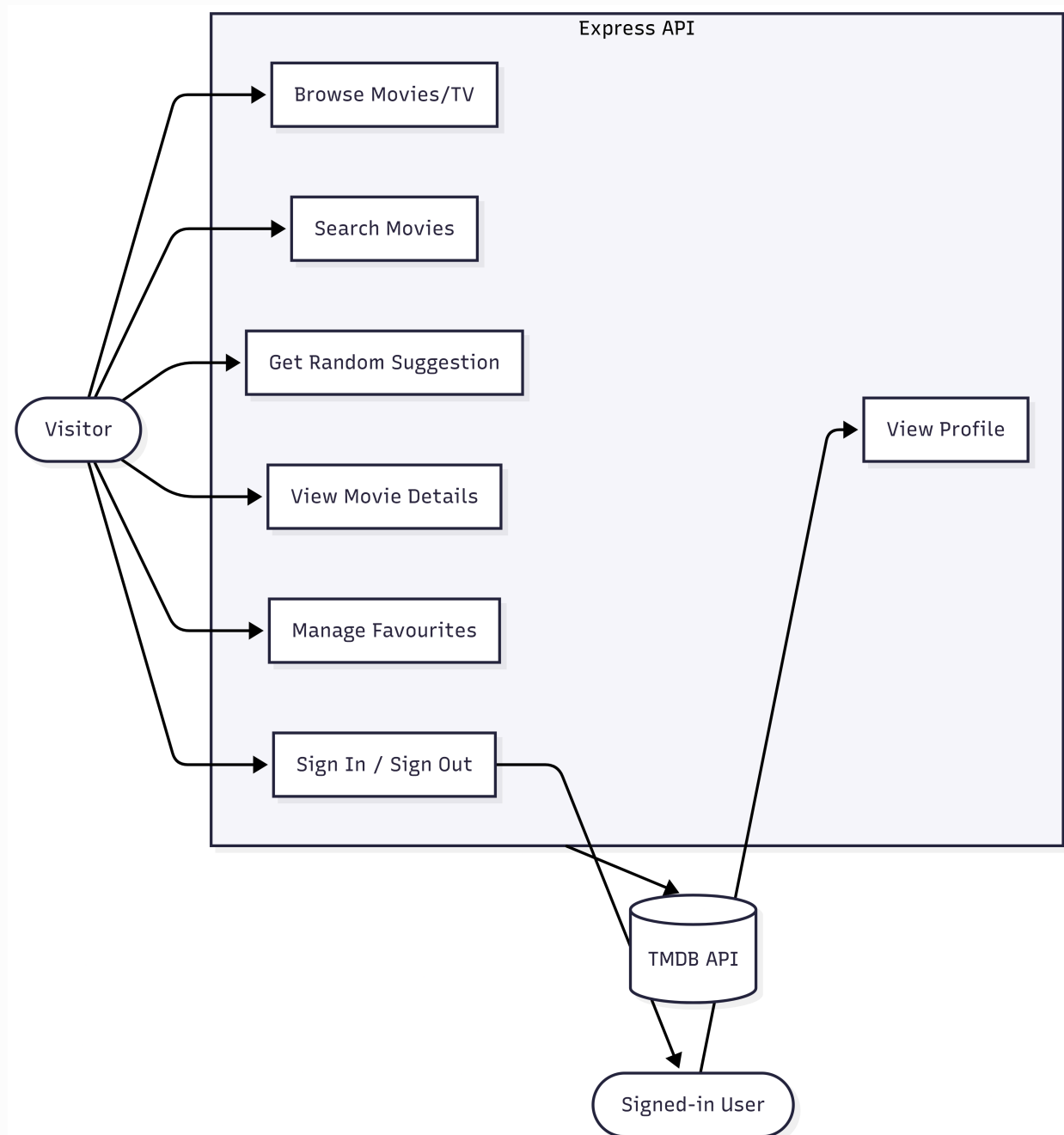
4. System Analysis & Design

4.1 Problem Statement & Objectives

Problem statement: Streaming platform users frequently experience "choice overload" when faced with massive content libraries, leading to decision fatigue and excessive browsing time rather than actual viewing.

Objectives: * Provide a fast, unified interface to discover movies/TV shows sourced from TMDb. * Reduce decision fatigue via a highly configurable, filter-driven random-pick generator. * Allow personalization through favorites and secure user authentication.

Use Case Diagram

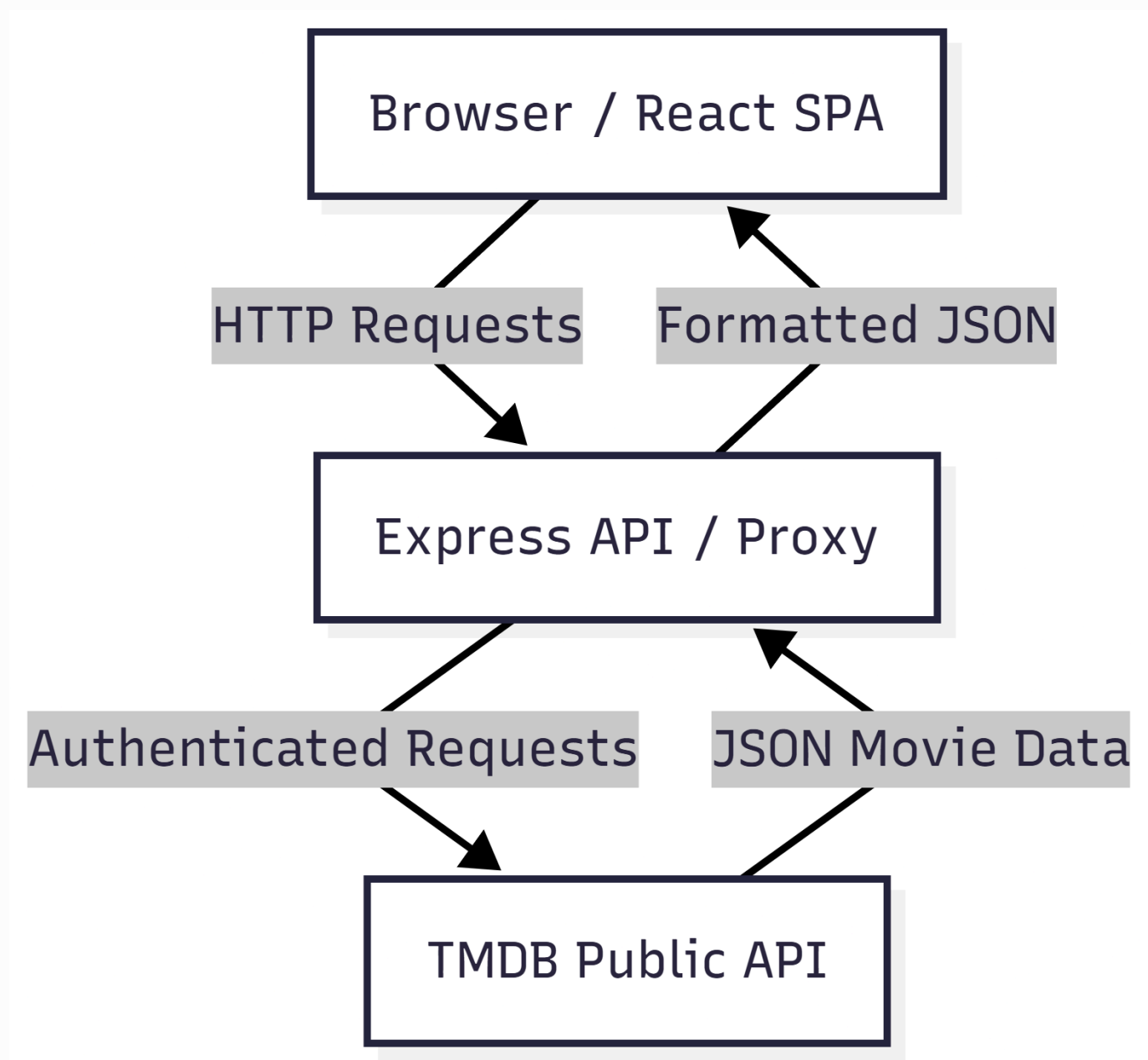


4.2 Database Design & Data Modeling

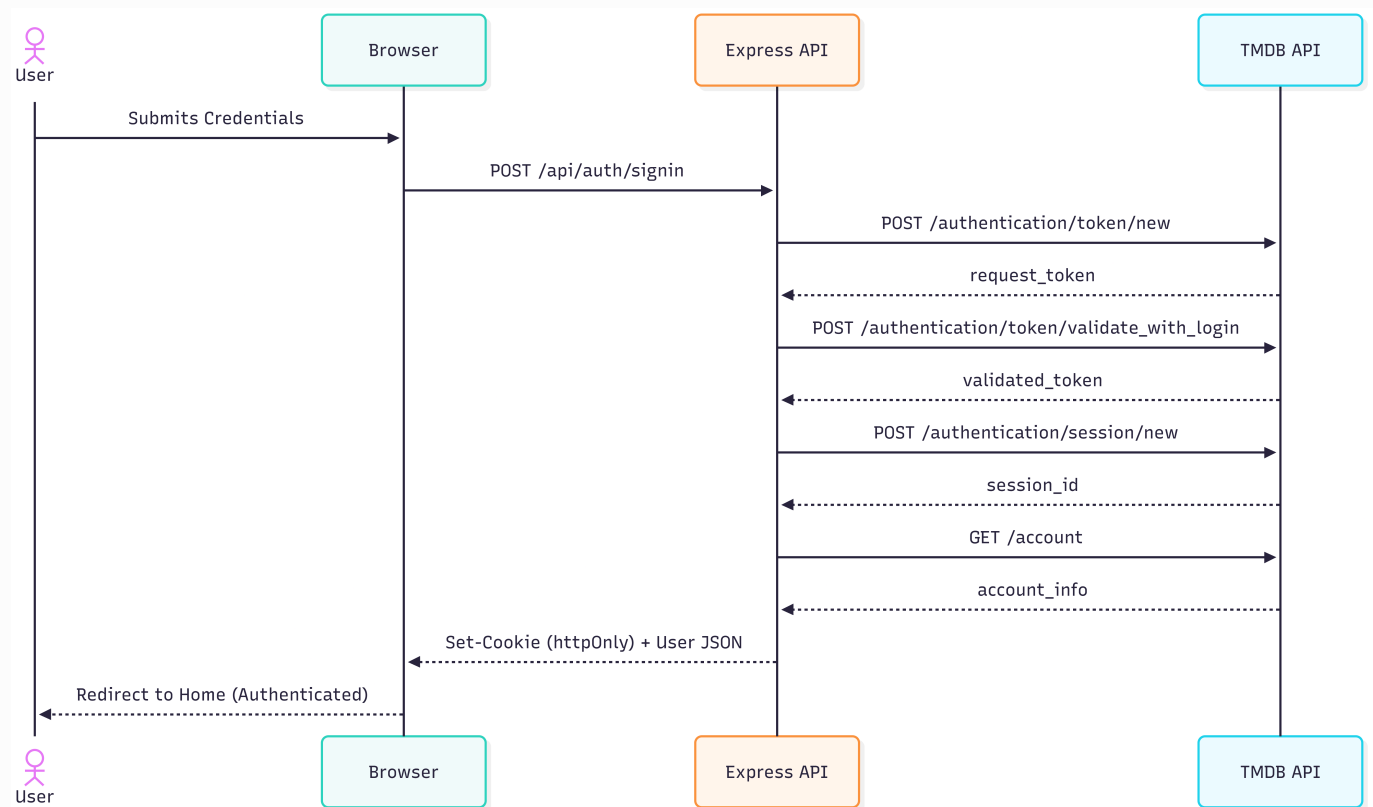
MovieRoulette employs a **stateless data architecture** regarding its catalog. It does not use an in-house relational or NoSQL database. * **Catalog Data:** TMDB API acts as the sole system of record. * **User Data (Favourites):** Stored client-side via browser `localStorage` under the key `movieroulette:favourites`. * **Authentication:** Managed via stateless JWTs / TMDB session cookies. This approach was intentionally chosen to maximize speed, minimize backend complexity, and respect the project's scope as a discovery layer rather than a data-hosting platform.

4.3 Data Flow & System Behavior

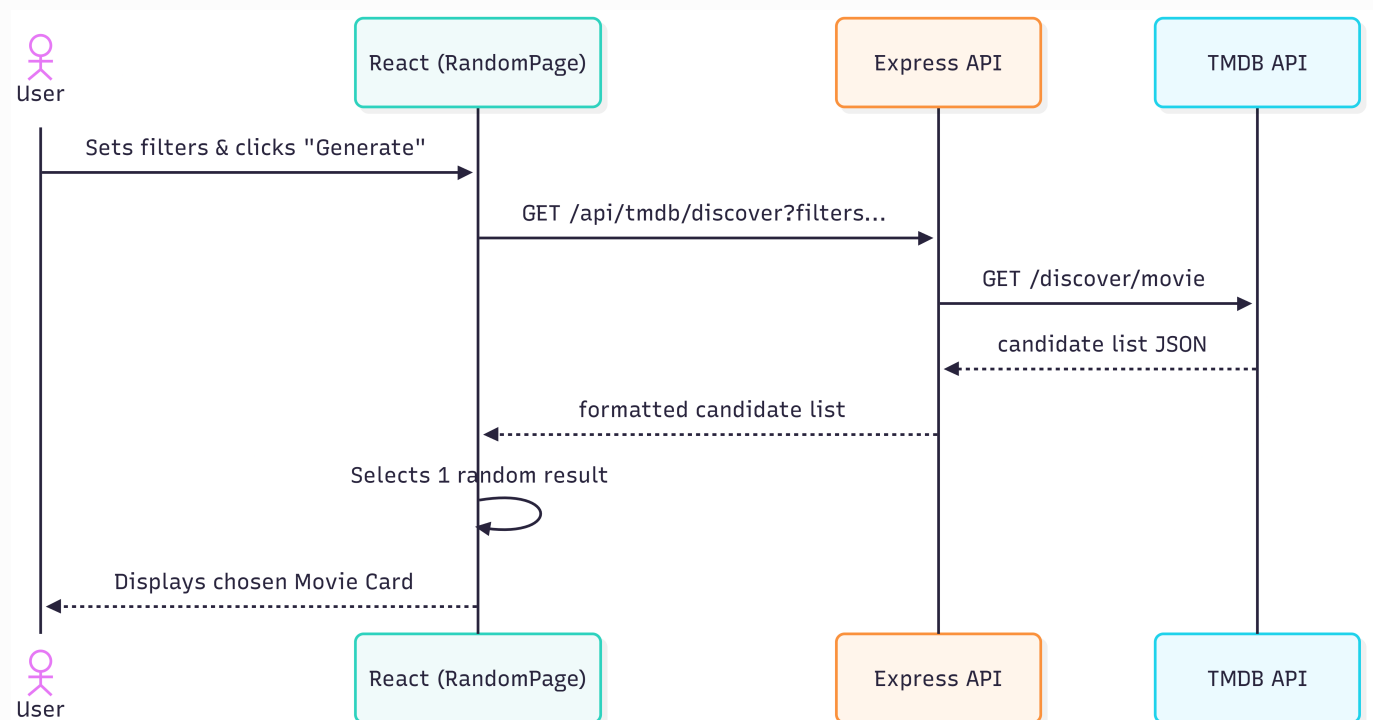
Context Data Flow Diagram (DFD)



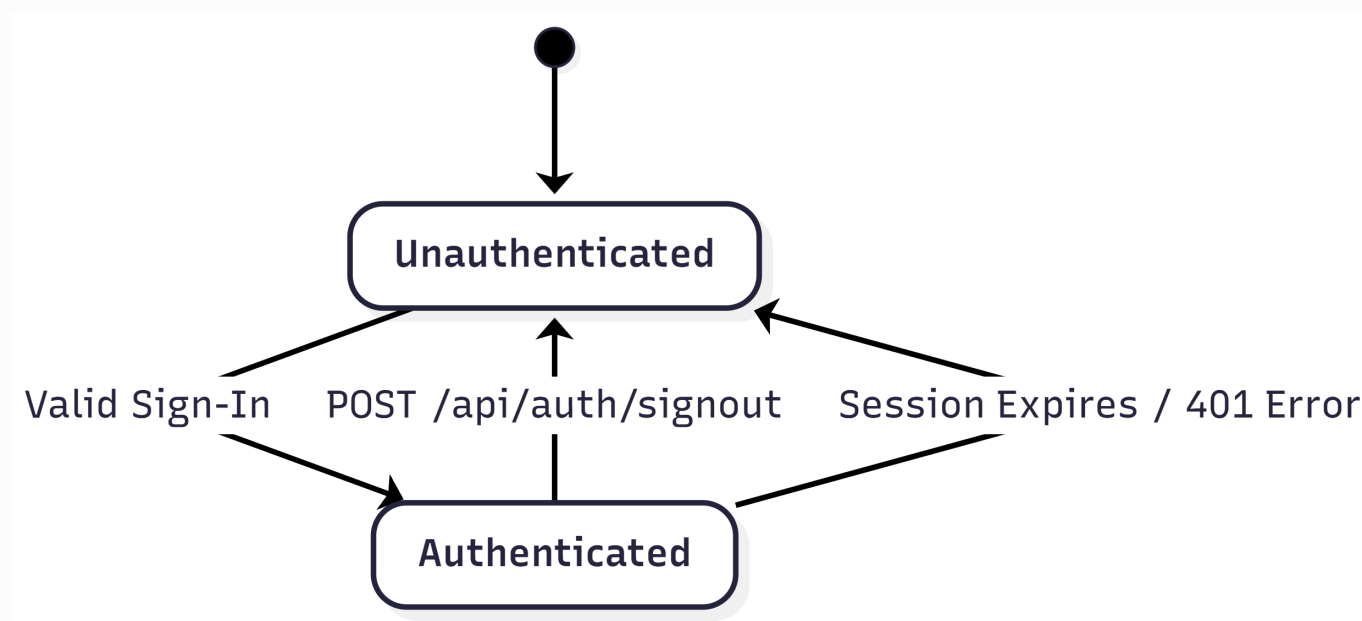
Sequence Diagram — Sign In Flow



Sequence Diagram — Random Movie Suggestion



State Diagram — Authentication State



4.4 UI/UX Design & Prototyping

UI/UX Guidelines:

- * **Component Library:** React-Bootstrap with standard Bootstrap 5 utility classes, customized via `styled-components`.
- * **Color Scheme:**
 - * Primary Accent: Coral (`#ff7668`) for interactive elements and highlights.
 - * Background: Dark cinematic theme (`#000000` to `#1a1a1a` surfaces).
- * **Typography:** 'Plus Jakarta Sans' for headers, 'Inter' for body text.
- * **Feedback Mechanisms:** Implementation of Skeletons (`Skeletons.tsx`) during API fetch periods to prevent layout shifts. Sonner toasts provide transient, non-blocking notifications.

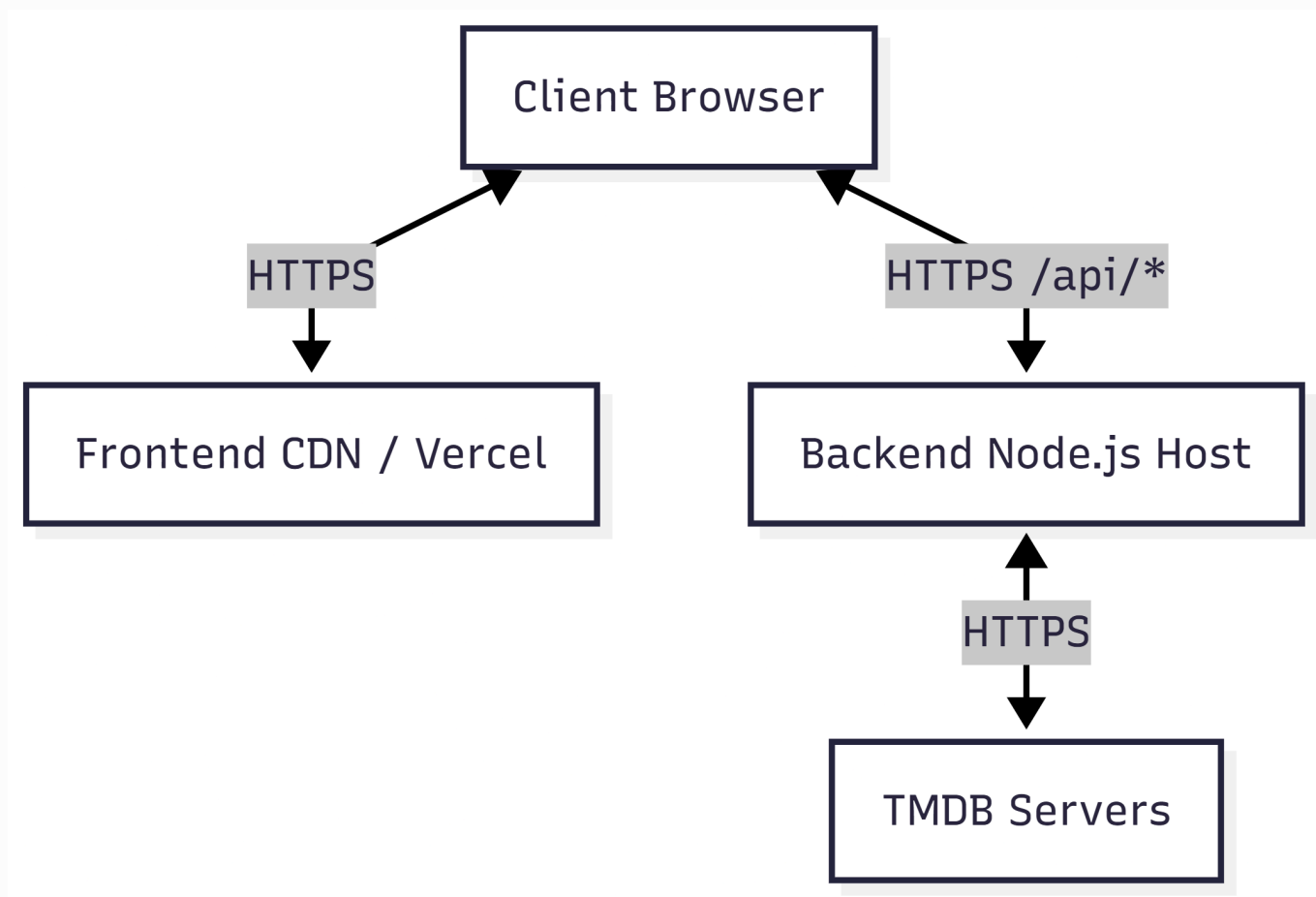
4.5 System Deployment & Integration

Technology Stack:

- * **Frontend:** React 19, Vite, TypeScript, Zustand (State), React Router v7.
- * **Backend:** Node.js, Express 5, cookie-parser, cors.
- * **External API:** TMDB REST API v3/v4.

Deployment Strategy: The architecture relies on a decoupled deployment model.

- * **Frontend:** The React SPA is built into static files via Vite (`npm run build`) and is optimized for CDN deployment platforms like Vercel, Netlify, or GitHub Pages.
- * **Backend:** The Express application runs on a Node.js compatible host (e.g., Heroku, Render, or a DigitalOcean Droplet).
- * **CI/CD:** Handled via a configured `Jenkinsfile` executing linting, testing, and production builds automatically upon merging to the main branch.



5. Implementation

5.1 Source Code Structure

The repository enforces strict separation of concerns: * `server/` : Express backend containing route handlers (`auth.routes.ts` , `tmdb.routes.ts`) and the TMDB client proxy. * `src/components/` : Presentational and layout UI components. * `src/features/` : Hooks and business logic organized by domain. * `src/store/` : Zustand global state configuration (`app-store.ts`). * `src/lib/` : Reusable TMDB client types, API interfaces, and utilities.

5.2 Version Control & Collaboration

- **Platform:** GitHub
- **Branching Strategy:** Feature Branching (GitFlow inspired). Features were developed on individual branches and merged to `main` via Pull Requests.
- **CI/CD Integration:** A Jenkins pipeline ensures code quality by running ESLint checks, Vitest executions, and simulated Vite builds prior to deployment.

5.3 Deployment & Execution Guide

Prerequisites: * Node.js (v18+) * A TMDB account for API credentials

Installation:

```
git clone <repository-url>
cd movie-roulette
npm install
```

Configuration: Create a `.env` file at the root directory:

```
TMDB_API_KEY=your_tmdb_v3_api_key
PORT=4000
CORS_ORIGIN=http://localhost:5173
```

Execution:

```
# Run both frontend and backend concurrently
npm run dev:full

# Or run them separately:
npm run server # Starts Express on port 4000
npm run dev    # Starts Vite on port 5173
```

6. Testing & Quality Assurance

6.1 Test Cases & Test Plan

Testing was conducted across multiple levels, targeting 80% coverage on core logic.

#	Scenario	Steps	Expected Outcome
1	Sign in with valid credentials	Navigate to <code>/signin</code> , enter valid TMDB credentials, submit.	Cookie is set, UI redirects to home, profile updates.
2	Sign in with invalid credentials	Submit incorrect password.	401 error returned; Toast displays "Incorrect credentials".
3	Random suggestion filtering	On <code>/random</code> , select 'Action', min-rating 7, and generate.	Returned movie matches 'Action' and rating ≥ 7 .
4	Toggle Favourite	Click the heart icon on a movie card.	Movie is instantly added to <code>/favourites</code> . Survives page reload.
5	Debounce Search	Type "Batman" rapidly in the search bar.	API is called only after the user stops typing (300ms delay).

6.2 Automated Testing

The project relies on Vitest for unit/integration testing and Storybook for UI component isolation.

```
# Execute unit tests
npx vitest run
```

Tests ensure that Zustand store state updates correctly when favorites are toggled and that API utility functions accurately parse TMDB payloads.

6.3 Bug Reports & Resolutions

- **CRIT-001: Random Generator Loop:** Under strict/impossible filter combinations, the random generator looped indefinitely. *Resolution:* Implemented a maximum retry count and graceful error messages ("No movies match your criteria").
- **HIGH-001: Mobile Menu Toggle:** Hamburger menu failed to open on iOS Safari. *Resolution:* Fixed CSS z-index and pointer-events in the navigation component.
- **HIGH-002: Search Lag:** Search input stuttered due to rendering delays. *Resolution:* Implemented a 300ms debounce hook.

7. Final Presentation & Reports

7.1 User Manual (Summary)

1. **Registration/Login:** Access the application, click "Sign In", and use your TMDB credentials to authenticate.
2. **Browsing:** Navigate to "Movies" or "Series" to view the catalog. Use the sidebar to filter by genre, year, or rating.
3. **Random Generator:** Click "Random" in the navigation. Set your mood parameters (e.g., "Horror", "1990-2000") and hit "Generate" to receive a tailored recommendation.
4. **Favorites:** Click the heart icon on any movie to save it. Access all saved titles via the "Favorites" tab.

7.2 Supplementary Materials

- **Technical Documentation:** Complete structural and architectural outlines are maintained within this document.
- **Project Presentation:** A formal slide deck (PPT/PDF) summarizing the problem, technical stack, and outcomes is included in the final submission archive.
- **Video Demonstration:** A screen-recorded walkthrough of the application's core user flows is provided alongside the presentation.